



DOSSIER DE CONCEPTION

Architecture de données intégrée OLTP, OLAP et NoSQL

Plateforme de paiement : transactions, analyse,
temps réel, détection de fraude et personnalisation

Projet Stripe

Diagrammes, modèles de données, pipeline, sécurité et requêtes métier

Aïcha Fathellah

Architecture de données pour l'intelligence artificielle

Octobre 2026

Version 1.0

Sommaire

1. Contexte et cahier des charges.....	4
1.1 Contexte.....	4
1.2 Besoins identifiés.....	4
1.3 Contraintes et normes.....	5
1.4 Hypothèses et périmètre de la démonstration.....	5
1.5 Correspondance entre les exigences et l'architecture.....	6
1.6 Livrables.....	7
2. Architecture globale.....	9
2.1 Vue d'ensemble.....	9
2.2 Rôle de chaque composant.....	11
2.3 Choix technologiques et alternatives écartées.....	11
2.4 Principes de conception.....	12
3. Base transactionnelle (OLTP).....	13
3.1 Modèle entité-relation.....	13
3.2 Normalisation et intégrité.....	15
3.3 Propriétés ACID et transactions multi-tables.....	15
3.4 Performances.....	16
3.5 Réplication en temps réel, reprise après sinistre et bascule.....	16
4. Entrepôt analytique (OLAP).....	17
4.1 Schéma en étoile.....	17
4.2 Dimensions et faits.....	19
4.3 Stratégie pour les jointures volumineuses, les sous-requêtes et les séries temporelles.....	19
4.4 Pré-agrégations et vues matérialisées.....	20
4.5 Alimentation incrémentale.....	20
5. Base NoSQL.....	21
5.1 Modèle de données.....	21
5.2 Collections.....	23
5.3 Exemples de documents.....	23
5.4 Imbriquer ou référencer.....	24
5.5 Validation et index.....	25
6. Pipeline de données.....	26
6.1 Architecture du pipeline.....	26
6.2 Traitement en temps réel.....	28
6.3 Traitements par lots.....	28
6.4 Garanties de livraison.....	29
6.5 Synchronisation et cohérence entre les bases.....	29
7. Intégration de l'intelligence artificielle.....	31
7.1 Cycle de vie des modèles.....	31
7.2 Intégration dans la base NoSQL.....	33
7.3 Extraction des indices.....	33
7.4 Détection de fraude en temps réel.....	33
7.5 Personnalisation et analyses prédictives.....	34

7.6 Suivi et mise à jour des modèles.....	34
8. Sécurité et conformité.....	36
8.1 Cadre de sécurité.....	36
8.2 Chiffrement.....	38
8.3 Contrôle d'accès par rôles.....	38
8.4 Journalisation d'audit.....	39
8.5 Conformité réglementaire.....	39
8.6 Rapport de conformité automatisé.....	40
9. Supervision, performances et évolutivité.....	41
9.1 Tableau de bord de supervision.....	41
9.2 Règles d'alerte.....	42
9.3 Identifier et optimiser : un exemple mesuré.....	42
9.4 Stratégies de performance.....	43
9.5 Montée en charge horizontale.....	43
10. Requêtes métier.....	44
10.1 Questions et bases interrogées.....	44
10.2 Analyse du chiffre d'affaires.....	45
10.3 Détection de fraude.....	45
10.4 Segmentation et personnalisation.....	46
10.5 Requête transverse : un même paiement dans les trois bases.....	46
11. Limites et architecture cible.....	48
11.1 Limites de la démonstration.....	48
11.2 Architecture cible dans le cloud.....	48
11.3 Serveurs virtuels et grappes de serveurs.....	49
Annexes.....	50
A. Organisation du dépôt.....	50
B. Séquence de déploiement.....	50
C. Glossaire.....	50

1. Contexte et cahier des charges

1.1 Contexte

Stripe est une entreprise de technologie financière fondée en 2010 à San Francisco. Sa plateforme de paiement permet à des entreprises de toutes tailles d'encaisser des paiements et de gérer leurs opérations financières. Elle traite des milliards de transactions par an pour des millions de marchands dans le monde.

Avec cette croissance, la gestion des données devient plus complexe : elles sont réparties entre des systèmes transactionnels (OLTP), analytiques (OLAP) et non relationnels (NoSQL), qui répondent chacun à un besoin différent. L'objectif est de concevoir une architecture de données unifiée et évolutive, capable de traiter de forts volumes de transactions, de répondre à des requêtes analytiques complexes et d'exploiter des données non structurées pour la détection de fraude, l'analyse du comportement des clients et les recommandations en temps réel.

Ce dossier présente l'architecture conçue pour répondre à ce besoin, puis sa mise en œuvre complète dans un environnement de démonstration : bases de données, pipeline de données, modèles d'apprentissage automatique, sécurité, supervision et requêtes métier.

1.2 Besoins identifiés

Le scénario décrit cinq difficultés rencontrées par Stripe au fil de son expansion internationale. Chacune se traduit par un besoin, auquel l'architecture apporte une réponse spécifique.

Difficulté du scénario	Besoin	Réponse retenue
Charges transactionnelles complexes : paiements, remboursements, litiges, abonnements	Haute disponibilité, cohérence, rapidité ; détection de fraude en temps réel	Base transactionnelle PostgreSQL normalisée, réplica en continu, capture des changements vers une notation en temps réel
Analyses avancées : chiffre d'affaires, segmentation, conformité, performance des produits	Requêtes complexes, analyses ad hoc, résultats en temps quasi réel	Entrepôt en étoile partitionné, agrégats pré-calculés, vues matérialisées, chargement toutes les 15 minutes
Données non structurées et semi-structurées : journaux, avis, navigation	Stockage flexible, interrogation des données imbriquées, intégration aux données relationnelles	Base MongoDB : sept collections validées, identifiants communs avec la base transactionnelle
Intégration entre les systèmes	Flux continu entre les trois bases, données cohérentes et à jour	Capture des changements (Debezium), diffusion (Kafka), orchestration des traitements par lots (Airflow)
Conformité et sécurité : RGPD, PCI-DSS, CCPA	Protection des données stockées et en transit, contrôle des accès, surveillance des usages	Chiffrement, contrôle d'accès par rôles, journal d'audit, 13 contrôles quotidiens, supervision continue

1.3 Contraintes et normes

Les données traitées sont des données financières et personnelles. Trois réglementations encadrent leur traitement, et chacune se traduit par des choix d'architecture précis.

Réglementation	Exigences principales	Traduction dans l'architecture
RGPD (Union européenne)	Minimisation (article 5) ; limitation de la conservation (article 5) ; protection dès la conception (article 25) ; sécurité du traitement (article 32) ; droit à l'effacement (article 17)	Entrepôt sans donnée personnelle directe ; journaux supprimés après 90 jours ; adresse IP exclue du flux temps réel ; chiffrement et contrôle d'accès ; effacement traité par pseudonymisation (chapitre 8)
PCI-DSS (secteur des cartes de paiement)	Aucun numéro de carte stocké (exigence 3) ; chiffrement des transmissions (exigence 4) ; restriction des accès (exigence 7) ; protection des identifiants (exigence 8) ; journalisation et surveillance (exigence 10)	Cartes remplacées par un jeton ; TLS 1.3 vers la base transactionnelle ; un compte par programme ; secrets hors du code ; journal d'audit non modifiable
CCPA (Californie)	Droit de savoir quelles données sont détenues ; droit de suppression ; droit de s'opposer à la vente des données	Identifiant client commun aux trois bases, permettant de retrouver et de traiter toutes les données d'une personne ; aucune donnée cédée à des tiers

S'y ajoutent des contraintes techniques et opérationnelles :

- **latence** : la décision de fraude doit être rendue avant que le paiement ne soit définitivement accepté ;
- **disponibilité** : une panne de la base transactionnelle ne doit entraîner ni arrêt prolongé ni perte de transactions ;
- **cohérence** : un même paiement doit être identique dans les trois bases, sans doublon ni perte ;
- **évolutivité** : l'architecture doit absorber la croissance des volumes sans refonte ;
- **traçabilité** : toute action sur les données sensibles doit pouvoir être retracée.

1.4 Hypothèses et périmètre de la démonstration

L'architecture a été entièrement mise en œuvre dans un environnement de démonstration : seize services Docker exécutés sur un seul poste (treize permanents, trois lancés à la demande : générateur, consommateur temps réel et entraînement initial), décrits dans un fichier `docker-compose.yml` versionné. Les données sont synthétiques et reproductibles : un générateur crée un historique réaliste, puis simule des paiements en continu.

Données de l'historique généré	Volume
Pays et devises de référence	17 pays, 11 devises
Taux de change quotidiens	4 400
Marchands et produits	50 marchands, 194 produits
Clients	2 000

Données de l'historique généré	Volume
Paielements (environ 13 mois)	42 536, dont 744 frauduleux simulés
Remboursements et litiges	1 224 remboursements, 446 litiges
Sessions de navigation et avis	24 399 sessions, 1 535 avis
Journaux techniques et documents reçus	3 000 journaux, 627 documents

En mode continu, le générateur produit un paiement toutes les deux secondes, soit environ 43 000 paiements par jour, contre une centaine par jour dans l'historique. Ce rythme volontairement accéléré rend le traitement en temps réel visible lors des démonstrations ; ses effets sur les analyses de la période en cours sont présentés au chapitre 11.

Le passage à une production réelle (services gérés dans le cloud, grappes de serveurs, chiffrement généralisé) est décrit dans l'architecture cible du chapitre 11.

1.5 Correspondance entre les exigences et l'architecture

Le tableau suivant reprend chaque exigence de l'énoncé et indique la réponse apportée, ainsi que le chapitre qui la détaille.

Exigence	Réponse apportée	Chapitre
OLTP : propriétés ACID, faible latence, transactions portant sur plusieurs enregistrements	PostgreSQL 17, schéma en troisième forme normale, contraintes et déclencheurs exécutés dans la même transaction	3
OLTP : réplication en temps réel, reprise après sinistre, bascule	Réplica alimenté en continu par réplication physique ; procédure de bascule par promotion du réplica	3
OLAP : schéma en étoile ou en flocon, agrégation sur plusieurs dimensions, séries temporelles	Schéma en étoile à sept dimensions, faits partitionnés par mois, fonctions de fenêtre	4
OLAP : jointures volumineuses et sous-requêtes, pré-agrégations, vues matérialisées	Clés de substitution entières, élagage des partitions, table d'agrégats quotidiens, trois vues matérialisées	4
OLAP : analyses historiques et en temps quasi réel	Chargement incrémental toutes les 15 minutes	4 et 6
NoSQL : données JSON, XML et binaires, schéma flexible, données imbriquées	MongoDB 8 : documents JSON validés, messages XML conservés et analysés, fichiers binaires dans GridFS	5
NoSQL : relations entre documents (imbrication, référence, index), intégration aux autres bases	Imbrication des données lues ensemble, références par identifiants communs, index composés, plein texte et d'expiration	5
Intégration : traitements par lots et en flux, synchronisation en temps réel, ETL et enrichissement	Capture des changements (Debezium), Kafka, consommateur temps réel, chaînes Airflow	6
Cohérence : capture des changements, cohérence à terme, résolution des conflits	Source de vérité unique, ordre garanti par client, écritures idempotentes, filigrane de chargement	6

Exigence	Réponse apportée	Chapitre
Évolutivité : montée en charge horizontale, index, partitionnement, mise en cache, répartition des données	Partitions Kafka et groupes de consommateurs, répartition MongoDB, réplicas de lecture, agrégats et vues comme cache	9
Sécurité : chiffrement, contrôle d'accès par rôles, journal d'audit	Chiffrement des e-mails et des échanges, neuf rôles et comptes de service, journal non modifiable	8
Conformité : RGPD, PCI-DSS, contrôles et rapports automatisés, surveillance des failles en temps réel	Rapport quotidien de 13 contrôles, 45 tests automatisés, tableau de bord et règles d'alerte	8 et 9
Apprentissage automatique : intégration dans la base NoSQL, fraude en temps réel, personnalisation, analyses prédictives	Indices et registre des modèles dans MongoDB, notation de chaque paiement, recommandations de produits	7
Apprentissage automatique : extraction des indices, déploiement, suivi et mise à jour des modèles	Code partagé entre entraînement et production, rechargement à chaud, suivi de dérive, réentraînement hebdomadaire	7
Requêtes SQL et NoSQL répondant aux questions métier	17 questions : chiffre d'affaires, fraude, segmentation, parcours d'un paiement dans les trois bases	10

1.6 Livrables

Livable demandé	Emplacement dans ce dossier	Fichiers du dépôt
Diagramme d'architecture global	Chapitre 2, figure 1	03_diagrammes/ 01_architecture_globale
Diagramme entité-relation de la base transactionnelle	Chapitre 3, figure 2	03_diagrammes/ 02_modele_entite_relation, 04_oltp_postgresql
Conception du schéma analytique	Chapitre 4, figure 3	03_diagrammes/03_schema_etoile, 05_olap_entrepot
Modèle de données NoSQL	Chapitre 5, figure 4	03_diagrammes/04_modele_nosql, 06_nosql_mongodb
Architecture du pipeline de données	Chapitre 6, figure 6	03_diagrammes/05_pipeline, 07_pipeline
Stratégie d'intégration de l'apprentissage automatique	Chapitre 7, figure 7	03_diagrammes/07_cycle_modeles, 07_pipeline/ml
Plan de sécurité et de conformité	Chapitre 8, figure 8	03_diagrammes/06_securite, 08_securite_conformite
Requêtes SQL et NoSQL	Chapitre 10	09_requetes_metier

En synthèse

Chaque exigence de l'énoncé est couverte par un composant mis en œuvre, testé et versionné. Les chapitres suivants détaillent la conception de chaque composant, les choix effectués et leurs justifications, puis les résultats mesurés.

2. Architecture globale

2.1 Vue d'ensemble

L'architecture repose sur un principe simple : chaque paiement naît dans la base transactionnelle, qui reste la source de vérité unique. Toute modification y est captée en temps réel, puis distribuée aux systèmes qui en ont besoin : le traitement temps réel pour la notation de la fraude et la personnalisation, la base NoSQL pour les données enrichies et non structurées, l'entrepôt analytique pour les analyses historiques.

Architecture globale de la plateforme de données

Base transactionnelle, capture des changements, traitement temps réel, base NoSQL, entrepôt analytique, orchestration et supervision

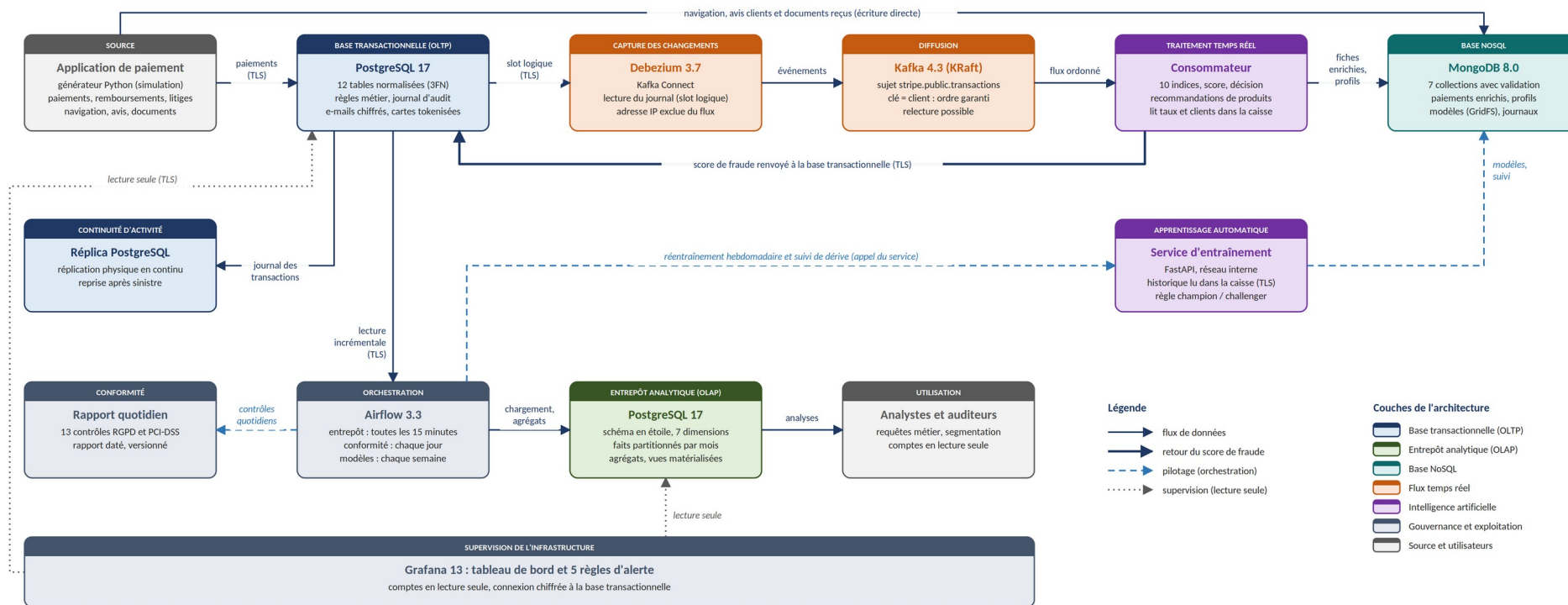


Figure 1 - Architecture globale de la plateforme de données

Comment lire ce diagramme

La rangée du haut suit un paiement en temps réel, de gauche à droite. La flèche épaisse en retour représente le score de fraude renvoyé à la base transactionnelle. La deuxième rangée regroupe la continuité d'activité (réplica) et l'apprentissage automatique. La troisième rangée correspond aux traitements par lots, orchestrés par Airflow. La rangée du bas montre la supervision. Les flèches pleines sont des flux de données, les tirets du pilotage, les pointillés de la lecture seule. La couleur de chaque bloc indique sa couche, rappelée dans la légende ; chaque bloc porte aussi son domaine en toutes lettres.

2.2 Rôle de chaque composant

Composant	Rôle	Version
PostgreSQL (base transactionnelle)	Enregistrement des paiements, remboursements, litiges et abonnements ; source de vérité	17
PostgreSQL (réplica)	Copie continue de la base transactionnelle, prête à prendre le relais	17
Debezium (Kafka Connect)	Capture des changements dans le journal des transactions	3.7
Apache Kafka	Diffusion ordonnée et rejouable des événements de paiement	4.3, mode KRaft
Consommateur temps réel (Python)	Calcul des indices, score de fraude, décision, recommandations	Python 3.12
MongoDB	Paiements enrichis, profils, registre des modèles, navigation, avis, journaux, documents	8.0
Service d'entraînement (FastAPI)	Entraînement, évaluation et suivi des modèles, appelé par Airflow	Python 3.12
Apache Airflow	Orchestration : alimentation de l'entrepôt, conformité, suivi des modèles	3.3.1
PostgreSQL (entrepôt analytique)	Schéma en étoile, agrégats et vues matérialisées pour les analyses	17
Grafana	Tableau de bord de supervision et règles d'alerte	13.2.3
Kafbat UI, Adminer, Mongo Express	Interfaces d'administration et de consultation	-

2.3 Choix technologiques et alternatives écartées

Chaque choix a été fait au regard des exigences de l'énoncé et comparé à des alternatives courantes. L'environnement de démonstration privilégie des logiciels libres, déployables localement et identiques à ceux proposés en services gérés dans le cloud.

Besoin	Choix	Alternatives écartées et raison
Base transactionnelle	PostgreSQL 17	MySQL : décodage logique et partitionnement moins complets. Bases SQL distribuées (CockroachDB, Spanner) : pertinentes à très grande échelle, retenues pour l'architecture cible (chapitre 11)

Besoin	Choix	Alternatives écartées et raison
Entrepôt analytique	PostgreSQL 17 en schéma en étoile	Entrepôts en colonnes (Snowflake, BigQuery, ClickHouse) : plus performants sur de très gros volumes, mais services externes ou supplémentaires ; même modèle en étoile transposable sans changement de conception
Base NoSQL	MongoDB 8.0	Cassandra : excellente en écriture massive mais requêtes moins souples sur des documents imbriqués. Elasticsearch : orienté recherche, moins adapté au stockage de référence
Capture des changements	Debezium	Interrogation périodique des tables : charge supplémentaire, latence plus forte, suppressions et états intermédiaires manqués
Diffusion des événements	Apache Kafka	RabbitMQ : pas de relecture d'un historique d'événements ni d'ordre garanti par clé à grande échelle
Orchestration	Apache Airflow	Planificateur cron : ni dépendances entre tâches, ni nouvelles tentatives, ni historique d'exécution
Supervision	Grafana	Interfaces propres à chaque outil : pas de vue unifiée ni de règle d'alerte transversale

2.4 Principes de conception

- **Source de vérité unique** : seule la base transactionnelle est modifiée par l'application ; les autres bases en sont dérivées.
- **Chaque base pour son usage** : intégrité transactionnelle pour l'OLTP, rapidité d'agrégation pour l'OLAP, souplesse de schéma pour le NoSQL.
- **Temps réel et lots complémentaires** : la capture des changements alimente la décision immédiate ; Airflow alimente les analyses toutes les 15 minutes.
- **Identifiants communs** : un paiement, un client ou un marchand porte le même identifiant dans les trois bases, ce qui permet de les relier.
- **Sécurité dès la conception** : un compte par programme, chiffrement, minimisation et journal d'audit sont intégrés au modèle, et non ajoutés après coup.
- **Infrastructure décrite en code** : seize services, rôles, règles et tableaux de bord sont versionnés et reproductibles.

3. Base transactionnelle (OLTP)

3.1 Modèle entité-relation

La base transactionnelle enregistre chaque opération financière au moment où elle se produit. Elle compte douze tables en troisième forme normale : chaque information n'est stockée qu'à un seul endroit, ce qui évite les incohérences lors des mises à jour.

Modèle entité-relation de la base transactionnelle

12 tables en troisième forme normale - PK : clé primaire, FK : clé étrangère, UQ : valeur unique - patte de corbeau : plusieurs, trait : un, cercle : facultatif

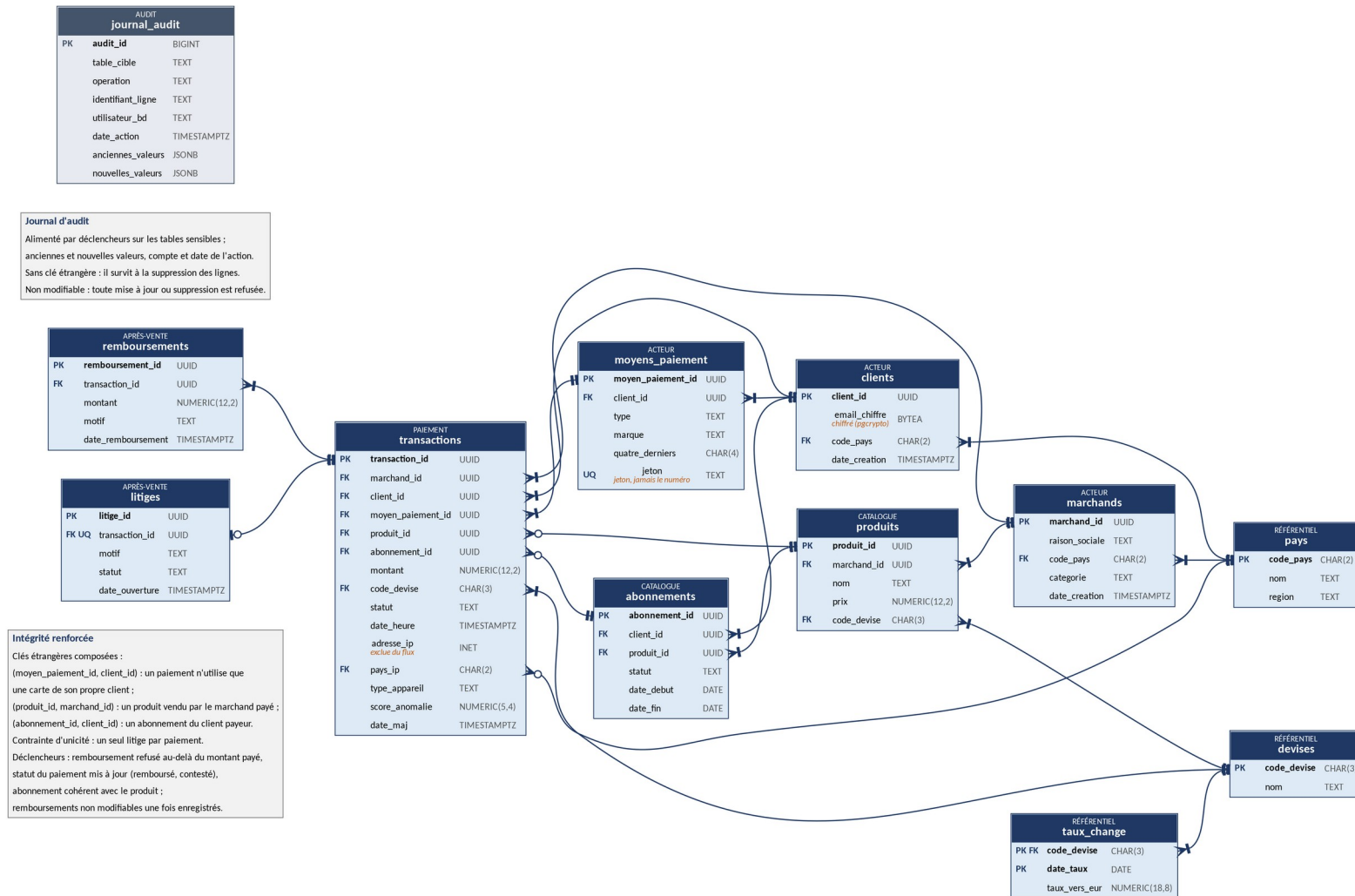


Figure 2 - Modèle entité-relation de la base transactionnelle

Comment lire ce diagramme

Chaque rectangle est une table ; son bandeau indique sa famille (référentiel, acteur, catalogue, paiement, après-vente, audit). PK désigne la clé primaire, FK une clé étrangère, UQ une valeur unique. Au bout de chaque trait, la patte de corbeau signifie « plusieurs », le trait « un », le cercle « facultatif ». Par exemple, un paiement porte au plus un produit (cercle, car un paiement d'abonnement n'en a pas), et un produit apparaît dans plusieurs paiements. Les mentions en orange signalent les colonnes sensibles.

3.2 Normalisation et intégrité

Famille	Tables	Rôle
Référentiels	pays, devises, taux_change	Données de référence stables : 17 pays, 11 devises, taux quotidiens vers l'euro
Acteurs	marchands, clients, moyens_paiement	Vendeurs, payeurs et leurs moyens de paiement
Catalogue	produits, abonnements	Offre des marchands et abonnements récurrents des clients
Paiement	transactions	Chaque paiement, son contexte (pays de l'adresse IP, appareil) et son score de fraude
Après-vente	remboursements, litiges	Remboursements partiels ou totaux, contestations
Audit	journal_audit	Trace de chaque création, modification ou suppression sur les tables sensibles

L'intégrité est garantie par la base elle-même, et non par les programmes qui l'utilisent :

- **contraintes de domaine (CHECK)** : montants strictement positifs, statuts limités à une liste de valeurs, codes pays et devises au bon format ;
- **clés étrangères composées** : un paiement ne peut utiliser qu'un moyen de paiement appartenant à son propre client, un produit vendu par le marchand payé, un abonnement du client payeur ;
- **contrainte d'unicité** : un seul litige par paiement ;
- **déclencheurs** (6 fonctions, 13 déclencheurs) : remboursement refusé au-delà du montant payé, statut du paiement mis à jour automatiquement (remboursé, contesté), cohérence entre abonnement et produit, remboursements et journal d'audit non modifiables, journalisation de chaque action sur les tables sensibles.

3.3 Propriétés ACID et transactions multi-tables

L'énoncé demande la prise en charge des « transactions multi-documents ». Dans une base relationnelle, cette notion correspond à une transaction qui modifie plusieurs tables en une seule opération indivisible. PostgreSQL garantit les quatre propriétés ACID :

Propriété	Signification	Mise en œuvre dans le projet
Atomicité	Tout ou rien	Un remboursement insère sa ligne et met à jour le statut du paiement dans la même transaction : si le contrôle échoue, rien n'est enregistré

Propriété	Signification	Mise en œuvre dans le projet
Cohérence	Les règles sont toujours respectées	Contraintes et déclencheurs vérifiés à chaque écriture
Isolation	Les opérations simultanées ne se gênent pas	Verrouillage de la ligne du paiement (<code>SELECT ... FOR UPDATE</code>) pendant le contrôle d'un remboursement ou d'un litige : deux remboursements simultanés ne peuvent pas dépasser le montant payé
Durabilité	Une transaction validée n'est jamais perdue	Journal des transactions (WAL) écrit avant la validation, puis répliqué

3.4 Performances

Treize index secondaires accélèrent les accès les plus fréquents, en plus des index créés automatiquement pour les clés primaires et les contraintes d'unicité :

- historique d'un client ou d'un marchand : index composés (client ou marchand, date décroissante) ;
- repérage des modifications récentes par l'alimentation de l'entrepôt : index sur la date de mise à jour ;
- **index partiels** : seuls les paiements concernés sont indexés (avec un produit, avec un abonnement, ou suspects avec un score d'au moins 0,8), ce qui réduit leur taille et leur coût de mise à jour ;
- clés étrangères des tables secondaires, et journal d'audit par table et par date.

La mémoire de cache de PostgreSQL a par ailleurs été portée à 512 Mo après détection d'un taux de lecture en cache insuffisant par la supervision (chapitre 9).

3.5 Réplication en temps réel, reprise après sinistre et bascule

Un réplica reçoit en continu le journal des transactions de la base principale (réplication physique en flux). Il est initialisé par une copie complète (`pg_basebackup`), puis rejoue chaque écriture quelques millisecondes après la base principale. Un compte dédié, `replicator`, est limité à ce seul usage, et la connexion est chiffrée.

La réplication est asynchrone : la base principale n'attend pas le réplica pour valider une transaction, ce qui préserve la latence. Le retard est surveillé en continu (chapitre 9) ; il était nul lors des mesures.

En cas de panne de la base principale, la **bascule** consiste à promouvoir le réplica en base principale (fonction `pg_promote()`), puis à rediriger les programmes vers lui. Dans l'environnement de démonstration, cette bascule est manuelle ; dans l'architecture cible, elle est automatisée par un gestionnaire de haute disponibilité (chapitre 11). PostgreSQL 17 permet en outre de synchroniser le slot de capture des changements sur le réplica, pour que Debezium reprenne sans perte après une bascule.

En synthèse

La base transactionnelle garantit l'intégrité par le modèle (troisième forme normale, contraintes, clés composées), par les propriétés ACID et par des déclencheurs exécutés dans la même transaction. Le réplica en continu assure la reprise après sinistre.

4. Entrepôt analytique (OLAP)

4.1 Schéma en étoile

L'entrepôt est conçu pour répondre rapidement aux questions d'analyse : chiffre d'affaires, fraude, segmentation, performance des produits. Il adopte un schéma en étoile : une table de faits centrale, dont chaque ligne est un paiement (le grain), entourée de sept dimensions qui décrivent son contexte.

Schéma en étoile de l'entrepôt analytique

Une table de faits (un paiement par ligne) entourée de sept dimensions - PK : clé primaire, FK : clé étrangère, UQ : clé naturelle de la base transactionnelle, M : mesure

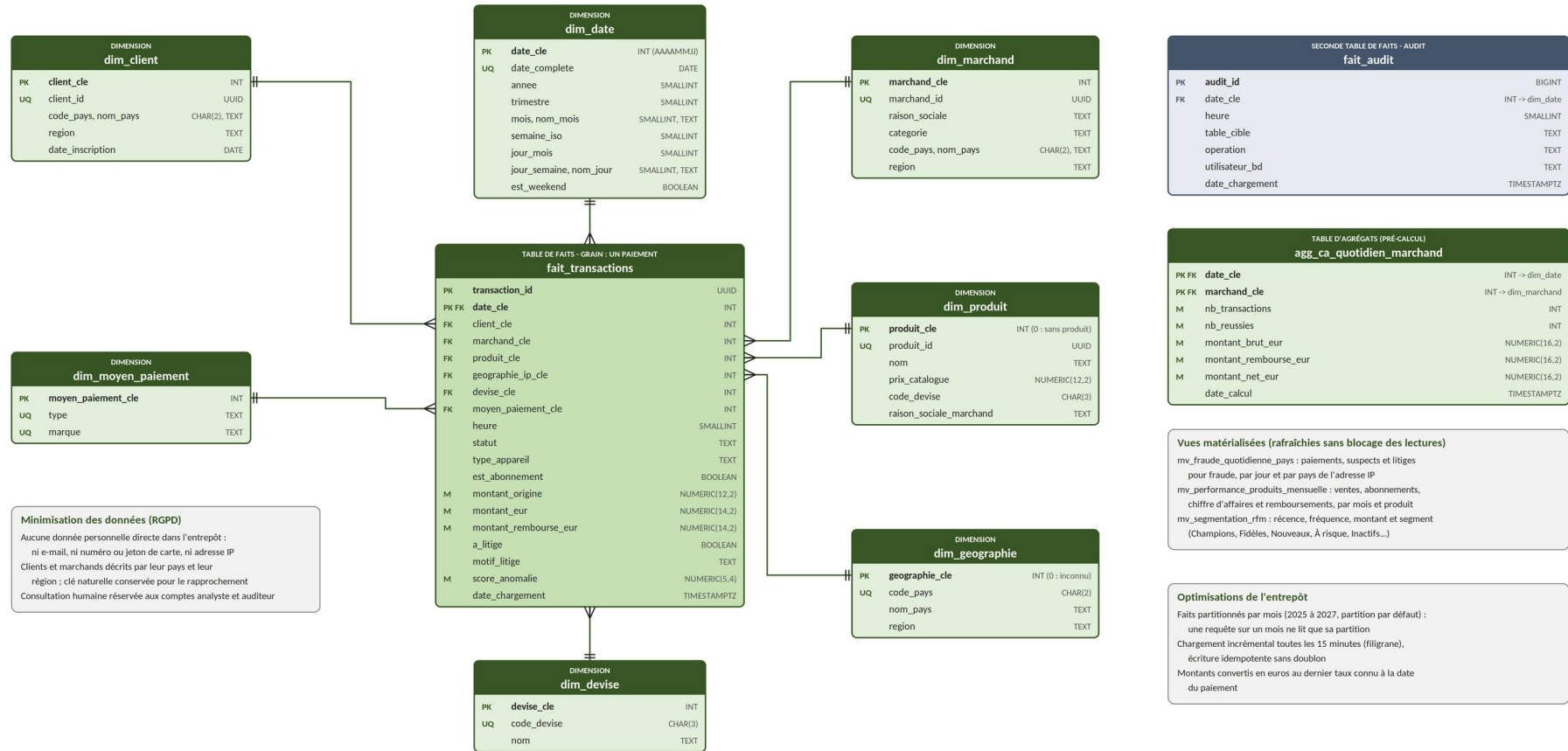


Figure 3 - Schéma en étoile de l'entrepôt analytique

Comment lire ce diagramme

La table de faits, au centre en vert soutenu, contient les mesures (repérées par M), c'est-à-dire les valeurs que l'on additionne ou compare. Les dimensions, en vert clair, permettent d'analyser « par » date, client, marchand, produit, pays, devise ou moyen de paiement. Chaque trait relie une clé étrangère de la table de faits à la clé primaire d'une dimension : une dimension (double trait) décrit plusieurs paiements (patte de corbeau). UQ désigne la clé naturelle venant de la base transactionnelle.

4.2 Dimensions et faits

Table	Contenu	Particularité
dim_date	Calendrier 2025 à 2027 : année, trimestre, mois, semaine, jour, week-end	Clé au format AAAAMMJJ, lisible et triable
dim_client	Pays, région, date d'inscription	Aucune donnée personnelle directe
dim_marchand	Raison sociale, catégorie, pays, région	-
dim_produit	Nom, prix catalogue, devise, marchand	Membre « sans produit » (clé 0) pour les paiements d'abonnement
dim_geographie	Pays et région de l'adresse IP	Membre « inconnu » (clé 0)
dim_devise	Code et nom de la devise	-
dim_moyen_paiement	Type et marque	Aucun numéro ni jeton de carte
fait_transactions	Un paiement par ligne : clés, heure, statut, appareil, montants, litige, score	Partitionnée par mois ; montants convertis en euros
fait_audit	Une action audité par ligne : table, opération, compte, date	Sans les valeurs : minimisation

Les dimensions utilisent des **clés de substitution** entières, générées par l'entrepôt. Elles sont plus compactes que les identifiants de la base transactionnelle (UUID), ce qui accélère les jointures, et elles isolent l'entrepôt des changements de la source. La clé d'origine est conservée (UQ) pour le rapprochement.

4.3 Stratégie pour les jointures volumineuses, les sous-requêtes et les séries temporelles

Difficulté	Stratégie retenue
Jointures entre une grande table de faits et les dimensions	Clés entières, index sur chaque clé étrangère de la table de faits, dimensions de petite taille gardées en mémoire
Volume croissant de la table de faits	Partitionnement mensuel par date (janvier 2025 à décembre 2027, plus une partition par défaut) : une requête sur un mois ne lit que sa partition (élagage)
Sous-requêtes et calculs intermédiaires	Expressions de table communes (WITH) et vues matérialisées qui pré-calculent les résultats intermédiaires coûteux

Difficulté	Stratégie retenue
Séries temporelles	Dimension date complète (semaine ISO, week-end) et fonctions de fenêtre : moyenne glissante, variation d'une période à l'autre
Questions fréquentes	Table d'agrégats quotidiens par marchand : quelques milliers de lignes lues au lieu de dizaines de milliers de paiements

4.4 Pré-agrégations et vues matérialisées

Objet	Grain	Mesures	Usage
agg_ca_quotidien_marchand	Jour et marchand	Nombre de paiements et de réussites, montants brut, remboursé et net	Chiffre d'affaires, classement des marchands
mv_fraude_quotidienne_pays	Jour et pays de l'adresse IP	Paiements, suspects, litiges pour fraude, montant suspect	Analyse géographique de la fraude
mv_performance_produits_mensuelle	Mois et produit	Ventes, abonnements, chiffre d'affaires, remboursements	Performance du catalogue
mv_segmentation_rfm	Client	Récence, fréquence, montant, segment	Segmentation : Champions, Fidèles, Nouveaux, À risque, Inactifs, À développer

Les vues matérialisées sont rafraîchies sans bloquer les lectures (`REFRESH MATERIALIZED VIEW CONCURRENTLY`), grâce à un index unique sur chacune. Le rafraîchissement passe par une fonction dédiée, exécutée avec les droits du propriétaire : le compte de chargement n'a pas besoin d'être propriétaire des vues.

4.5 Alimentation incrémentale

L'entrepôt est alimenté toutes les 15 minutes par Airflow, ce qui fournit des analyses en temps quasi réel. Chaque passage ne relit que les paiements modifiés depuis le dernier chargement, repérés par un **filigrane** (la date de la dernière modification chargée), avec une fenêtre de recouvrement de 5 minutes par sécurité. L'écriture est **idempotente** (`INSERT ... ON CONFLICT DO UPDATE`) : relire un paiement déjà chargé le met à jour sans créer de doublon. Après chaque chargement, des contrôles de qualité vérifient la complétude, la validité des montants et la cohérence entre agrégats et faits (chapitre 6).

Lors des mesures, l'entrepôt contenait 44 115 paiements, 96 120 actions auditées, 15 817 lignes agrégées et 1 984 clients segmentés ; le premier chargement complet a pris 22 secondes, les chargements incrémentaux quelques secondes.

5. Base NoSQL

5.1 Modèle de données

La base NoSQL (MongoDB) accueille les données dont la structure varie ou s'imbrique : paiements enrichis par le modèle, profils des clients, registre des modèles, navigation, avis, journaux techniques et documents reçus de l'extérieur. Elle traite les trois types de données cités par l'énoncé : JSON (les documents eux-mêmes), XML (messages bancaires conservés tels que reçus) et binaires (fichiers stockés dans GridFS).

Modèle de données NoSQL (MongoDB 8.0)

Sept collections avec règles de validation, deux espaces de fichiers GridFS - _id : identifiant, REF : référence, {} : objet imbriqué, [] : liste imbriquée

PAIEMENT NOTÉ EN TEMPS RÉEL transactions_enrichies		
_id	_id	id du paiement (caisse)
REF	client_id	vers profils_clients
REF	marchand_id	vers caisse : marchands
	montant	décimal (Decimal128)
	devise, statut	chaîne, énumération
	date_heure	date
{}	contexte	objet
	montant_eur, pays_ip, type_appareil	
{}	indicateurs	objet : 10 indices
{}	score	objet
	valeur, modele, version	
	decision	accepter, verifier, bloquer
{}	historique_statuts	liste (20 au plus)
	statut, date	
	latence_ms	entier
	date_traitement	date

HABITUDES ET PERSONNALISATION profils_clients		
_id	_id	id du client (caisse)
	code_pays, date_inscription	chaîne, date
	nb_paiements	entier
	montant_total_eur, montant_moyen_eur	décimal
[]	pays_connus	codes pays
[]	appareils_connus	types d'appareils
[]	marchands_connus	identifiants
[]	horodatages_recents	dates (24 heures)
[]	produits_achetes	identifiants
[]	categories_preferes	catégorie : nombre
[]	recommandations	liste (5 produits)
	produit_id, score, origine	
{}	recommandations_modele	nom, version
	date_maj	date

REGISTRE DES MODÈLES modeles_ml		
_id	_id	ObjectId
	nom, version	unique (index)
	statut	candidat, actif, archive
	date_entrainement	date
	seuil_verification, seuil_decision	0.5 et 0.8
{}	periode	début, fin des données
{}	metriques	objet
	entrainement, production, suivi	AUC, latence, dérive
REF	fichier_gridfs_id	vers GridFS « modeles »

JOURNAUX TECHNIQUES journaux_applicatifs		
_id	_id	ObjectId
	horodatage	date (expiration 90 j)
	niveau	DEBUG à CRITIQUE
{}	source	objet
	service, hote	chaîne
	message	chaîne
	code_http, duree_ms	nombre facultatifs

Imbriquer ou référencer

Imbriqué : ce qui est lu ensemble, en une seule lecture (score, indices, événements d'une visite, historique)
Listes bornées (20, 500) : un document ne grossit pas sans limite
Référéncé : ce qui vit ailleurs ou grossit (client, marchand, fichiers binaires)
Identifiants communs avec la caisse : écritures idempotentes, liens entre bases

Validation et index

7 collections avec règles \$jsonSchema :
champs obligatoires, types, valeurs admises ; champs supplémentaires admis
Structures variables : avis ou enquête, message texte ou fichier (anyOf)
Index composés client et date, unicité nom et version des modèles, recherche plein texte en français, expiration à 90 j

VISITES SUR LES SITES MARCHANDS sessions_navigation		
_id	_id	id de session
REF	client_id	facultatif : visiteur anonyme
REF	marchand_id	vers caisse : marchands
	debut, fin	date
{}	appareil	objet : type
[]	evenements	liste (500 au plus)
	type, horodatage, page	page vue, panier, paiement...
REF	transaction_id	si la visite aboutit

AVIS ET ENQUÊTES (STRUCTURES VARIABLES) avis_clients		
_id	_id	ObjectId
	type	avis ou enquête
REF	client_id, marchand_id	vers caisse
	date	date
	note	1 à 5 (si avis)
	commentaire	recherche plein texte
[]	reponses	si enquête
	question, reponse	

MESSAGES BANCAIRES ET JUSTIFICATIFS documents_externes		
_id	_id	ObjectId
	type	message ou pièce
	format	xml, json, pdf, image
	date_reception	date
{}	references	objet
REF	transaction_id, litige_id	vers caisse
	contenu_brut	si XML ou JSON
{}	champs_extraits	motif, montant...
REF	fichier_gridfs_id	si PDF ou image

STOCKAGE DE FICHIERS BINAIRES GridFS « fichiers » et « modeles »		
fichiers.files, fichiers.chunks	justificatifs PDF, images	
modeles.files, modeles.chunks	modèles entraînés	
découpage en blocs de 255 Ko	sans limite de 16 Mo	

Sécurité

Un compte par programme, droits définis collection par collection
Aucun droit de suppression accordé
Aucune donnée de carte ni adresse IP
Journaux supprimés après 90 jours (RGPD)

Figure 4 - Modèle de données NoSQL

Comment lire ce diagramme

Chaque bloc est une collection, l'équivalent d'une table. Les champs décalés vers la droite sont imbriqués dans le champ qui les précède : tout est stocké dans un seul document, lu en une seule fois. Les symboles indiquent l'identifiant (`_id`), une référence (REF), un objet imbriqué (`{ }`) ou une liste imbriquée (`[ ]`). Aucun trait ne relie les collections : en NoSQL, une référence est un identifiant recopié, que la base ne vérifie pas ; la cohérence des références est assurée par les programmes, qui utilisent les mêmes identifiants que la base transactionnelle.

5.2 Collections

Collection	Contenu	Écrite par
transactions_enrichies	Paieement noté : contexte, 10 indices, score, décision, latence, historique des statuts	Consommateur temps réel
profils_clients	Habitudes du client (pays, appareils, marchands connus, montants) et recommandations	Consommateur temps réel, entraînement
modeles_ml	Registre des versions : statut, seuils, métriques, référence du fichier du modèle	Service d'entraînement
sessions_navigation	Visites : appareil, liste des événements (page vue, panier, paiement)	Application
avis_clients	Avis notés ou réponses à des enquêtes, de structures différentes	Application
journaux_applicatifs	Journaux techniques, supprimés automatiquement après 90 jours	Tous les services
documents_externes	Messages bancaires XML et justificatifs PDF liés aux litiges	Application

5.3 Exemples de documents

Fiche d'un paiement noté en temps réel (valeurs réelles relevées pendant la démonstration, identifiants abrégés) :

```
{
  "_id": "613c0ea8-...",          // identifiant du paiement dans la base transactionnelle
  "client_id": "...",
  "marchand_id": "...",
  "montant": NumberDecimal("131.77"),
  "devise": "EUR",
  "statut": "reussie",
  "date_heure": ISODate("2026-10-04T19:17:12Z"),
  "contexte": {
    "type_appareil": "mobile", "pays_ip": "BE", "montant_eur": NumberDecimal("131.77")
  },
  "indicateurs": {
    "montant_eur_log": 4.89, "ecart_au_montant_moyen": 0.79, "pays_ip_inhabituel": 0,
    "appareil_inhabituel": 0, "marchand_nouveau": 0, "nb_paiements_derniere_heure": 1,
    "heure": 19, "est_nuit": 0, "anciennete_jours": 529, "nb_paiements_anterieurs": 43
  },
  "score": { "valeur": 0.8298, "modele": "fraude_hgb", "version": 1, "decision": "bloquer" },
  "latence_ms": 711,
  "historique_statuts": [
    { "statut": "reussie", "date": ISODate("2026-10-04T19:17:12Z") }
  ]
}
```

Message bancaire XML conservé tel que reçu, avec ses informations utiles extraites pour être interrogeables :

```
{
  "type": "message_bancaire",
  "format": "xml",
  "references": { "transaction_id": "...", "litige_id": "..." },
  "contenu_brut": "<NotificationLitige>
    <Reference>...</Reference><Transaction>...</Transaction>
    <Montant devise=\"EUR\">131.77</Montant><Motif>fraude</Motif>
    <Date>2026-10-04</Date>
  </NotificationLitige>",
  "champs_extraits": { "montant": 131.77, "devise": "EUR", "motif": "fraude" }
}
```

Un justificatif PDF est stocké dans GridFS, qui découpe les fichiers en blocs de 255 Ko et lève ainsi la limite de 16 Mo d'un document ; le document de la collection n'en garde que la référence (fichier_gridfs_id).

5.4 Imbriquer ou référencer

Choix	Critère	Exemples
Imbriquer	Données lues ensemble, appartenant au document, de taille bornée	Score et indices d'un paiement ; événements d'une visite (500 au plus) ; historique des statuts (20 au plus)
Référencer	Données partagées, vivant ailleurs ou volumineuses	Client et marchand (base transactionnelle) ; fichiers binaires (GridFS) ; paiement lié à une visite

Les listes imbriquées sont bornées par les règles de validation, pour qu'un document ne grossisse jamais sans limite. Les références utilisent les identifiants de la base transactionnelle : c'est ce qui permet d'intégrer la base NoSQL à l'OLTP et à l'OLAP, comme le montre la requête transverse du chapitre 10.

5.5 Validation et index

Les sept collections disposent de règles de validation (`$jsonSchema`) : champs obligatoires, types, valeurs admises et bornes. Les structures variables sont décrites par des alternatives (`anyOf`) : un avis exige une note, une enquête des réponses ; un message XML exige son contenu, un fichier sa référence GridFS. Quatorze tests automatisés vérifient que les documents conformes sont acceptés et les autres refusés.

Index	Collection	Rôle
Composés client et date, marchand et date	transactions_enrichies, sessions, avis	Historique d'un client ou d'un marchand
Partiel sur les scores d'au moins 0,8	transactions_enrichies	Accès rapide aux paiements suspects
Unique nom et version ; unique partiel sur la version active	modeles_ml	Une seule version active par modèle, garantie par la base
Plein texte en français	avis_clients	Recherche dans les commentaires, formes fléchies comprises
Expiration à 90 jours (TTL)	journaux_applicatifs	Limitation de la conservation (RGPD)
Références transaction et litige	documents_externes	Rassemblement des pièces d'un dossier de litige

Mongo Express Database: stripe_nosql Collection: transactions_enrichies

Viewing Collection: transactions_enrichies

New Document New Index

Simple Advanced

Key

Value

String

Find

Delete all 275 documents retrieved

1 2 3 4 5 > >>

_id	client_id	contexte	date_heure	date_traitement	devise	historique_statuts
fa0b7b46-ecf7-4ef8-9f84-5341e6ec24c	06e6bc14-f1a3-479d-827a-a60f999622e9	"type_appareil": "mobile", "pays_ip": "US", "produit_id": "e079d334-71f1-453a-a4f4-e3091440a40a", "abonnement_id": null, "montant_eur": @{1 item}	Sun Oct 04 2026 16:27:10 GMT+0000 (Coordinated Universal Time)	Sun Oct 04 2026 16:57:10 GMT+0000 (Coordinated Universal Time)	USD	"statut": "payé", "date": "2026-10-04"
9a64395e-3eee-4ec5-8222-1ae3b2d7f04	5b388f3b-e260-4573-a418-e2b5168d027b	"type_appareil": "mobile", "pays_ip": "FR", "produit_id": null, "abonnement_id": null, "montant_eur": @{1 item}	Sun Oct 04 2026 16:27:12 GMT+0000 (Coordinated Universal Time)	Sun Oct 04 2026 16:57:11 GMT+0000 (Coordinated Universal Time)	EUR	@{2 items}

Figure 5 - Fiches enrichies dans la base NoSQL (Mongo Express)

6. Pipeline de données

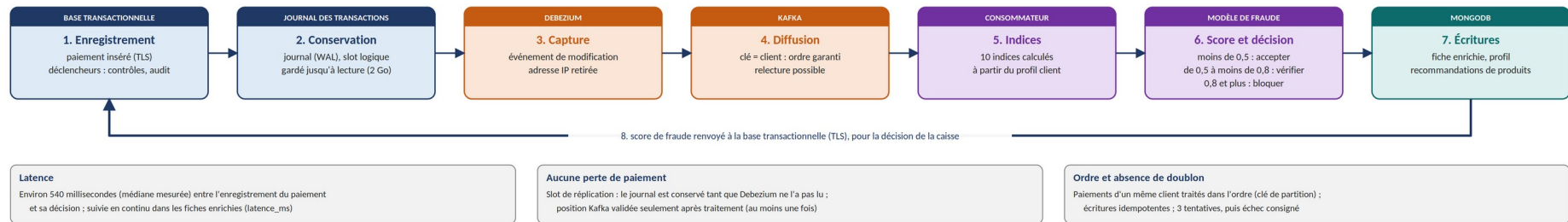
6.1 Architecture du pipeline

Le pipeline combine deux modes complémentaires : un flux en temps réel, qui traite chaque paiement en moins d'une seconde, et des traitements par lots orchestrés par Airflow, qui alimentent l'entrepôt, contrôlent la conformité et entretiennent les modèles.

Architecture du pipeline de données

Traitement en temps réel de chaque paiement, puis traitements par lots orchestrés par Airflow

Parcours d'un paiement en temps réel



Traitements par lots orchestrés par Airflow 3.3

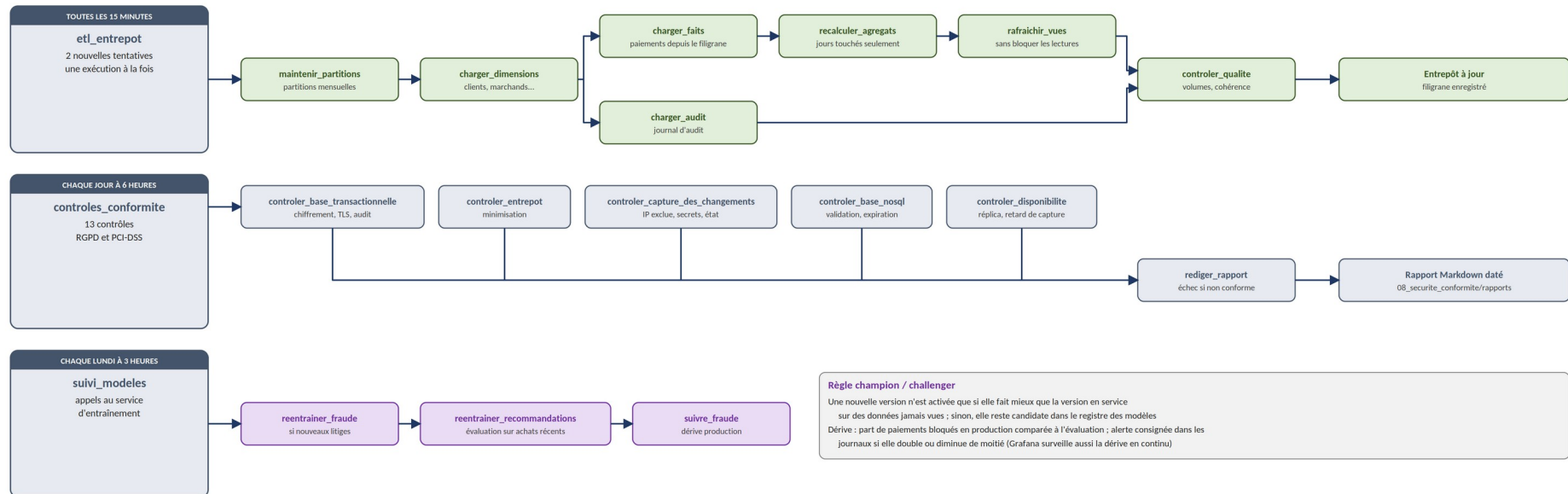


Figure 6 - Architecture du pipeline de données

Comment lire ce diagramme

En haut, les huit étapes se lisent de gauche à droite ; leurs couleurs reprennent celles des couches. La flèche épaisse en retour est l'étape 8. En bas, chaque couloir est une chaîne Airflow : à gauche sa fréquence, à droite ses tâches avec leur nom exact dans le code. Une tâche ne démarre que lorsque celles qui la précèdent ont réussi ; les tâches placées l'une au-dessus de l'autre s'exécutent en parallèle.

6.2 Traitement en temps réel

Étape	Outil	Fonctionnement
Capture	Debezium (connecteur PostgreSQL, décodage logique pgoutput)	Lit le journal des transactions par un slot de réplication dédié ; une publication limite la capture à la table des paiements ; l'adresse IP est exclue de l'événement
Diffusion	Kafka, sujet <code>stripe.public.transactions</code>	La clé de chaque événement est l'identifiant du client : tous les paiements d'un même client vont dans la même partition et sont traités dans l'ordre
Traitement	Consommateur Python (groupe <code>consommateur-temps-reel</code>)	Calcule les 10 indices à partir du profil du client, note le paiement, décide (accepter, vérifier, bloquer), met à jour le profil et les recommandations
Écritures	MongoDB et base transactionnelle	Fiche enrichie et profil dans MongoDB ; score renvoyé à la base transactionnelle par une connexion chiffrée

La latence médiane mesurée entre l'enregistrement d'un paiement et sa décision est d'environ 540 millisecondes. Les changements de statut (remboursement, litige) sont eux aussi captés et ajoutés à l'historique des statuts de la fiche.

6.3 Traitements par lots

Chaîne Airflow	Fréquence	Tâches	Résultat
etl_entrepot	Toutes les 15 minutes ; 2 nouvelles tentatives ; une exécution à la fois	Partitions, dimensions, puis en parallèle faits, agrégats et vues d'une part, audit d'autre part ; contrôle de qualité	Entrepôt à jour, filigrane enregistré
contrroles_conformite	Chaque jour à 6 heures	Cinq familles de contrôles en parallèle, puis rédaction du rapport	Rapport daté de 13 contrôles ; échec de la chaîne si un contrôle n'est pas conforme
suivi_modeles	Chaque lundi à 3 heures	Réentraînement de la fraude, puis des recommandations, puis suivi de la dérive	Nouvelles versions candidates ou activées ; alerte de dérive

Les étapes d'extraction, de transformation et de chargement (ETL) incluent la conversion des montants en euros, la résolution des clés de dimensions, l'enrichissement par le litige et le montant remboursé, et la minimisation : aucune donnée personnelle directe n'est copiée dans l'entrepôt, où les clients ne sont désignés que par un identifiant technique (pseudonyme).

6.4 Garanties de livraison

Garantie	Mécanisme
Aucune perte	Le slot de réplication conserve le journal tant que Debezium ne l'a pas lu (plafond de 2 Go pour protéger le disque). La position de lecture dans Kafka n'est validée qu'après le traitement complet d'un paiement : livraison « au moins une fois »
Aucun doublon	Écritures idempotentes : la fiche enrichie porte l'identifiant du paiement comme clé ; un paiement traité deux fois écrase sa propre fiche
Ordre	Clé de partition Kafka égale au client : les habitudes d'un client sont mises à jour dans l'ordre de ses paiements
Résilience	Trois tentatives par message, puis consignation de l'échec dans les journaux techniques ; nouvelles tentatives automatiques dans Airflow

Ces garanties ont été vérifiées en conditions réelles : après un redémarrage de la base transactionnelle, la tâche de capture s'est arrêtée pendant que le générateur continuait d'enregistrer des paiements ; à sa relance, Debezium est reparti de sa dernière position et les paiements en attente ont été transmis puis notés, avec une latence médiane de 21,5 secondes pendant ce rattrapage, avant un retour à environ 0,5 seconde.

6.5 Synchronisation et cohérence entre les bases

Les trois bases ne sont pas mises à jour au même instant : l'architecture adopte une **cohérence à terme** (eventual consistency), où chaque base dérivée rejoint l'état de la source dans un délai connu et surveillé.

Base	Délai de cohérence	Mécanisme
Base transactionnelle	Immédiat (source de vérité)	Transactions ACID
Réplica	Quelques millisecondes	Réplication physique en flux
Base NoSQL	Environ 0,5 seconde	Capture des changements, Kafka, consommateur
Entrepôt analytique	15 minutes au plus	Chargement incrémental par filigrane

La **résolution des conflits** repose sur des règles simples et déterministes :

- **un seul écrivain par donnée** : l'application écrit les paiements, le consommateur le seul score, Airflow l'entrepôt ; aucune donnée n'est modifiée par deux sources concurrentes ;
- **la source fait foi** : en cas d'écart, l'entrepôt et la base NoSQL sont réécrits à partir de la base transactionnelle, grâce aux écritures idempotentes et à la relecture possible des événements Kafka ;
- **la version la plus récente l'emporte** : l'entrepôt retient l'état du paiement associé à la date de mise à jour la plus récente ;
- **contrôles de réconciliation** : après chaque chargement, le nombre de paiements de l'entrepôt est comparé à celui de la source, et les agrégats aux faits ; tout écart fait échouer la chaîne.

En synthèse

Le pipeline traite chaque paiement en temps réel, sans perte, sans doublon et dans l'ordre, tout en alimentant l'entrepôt par lots. La cohérence à terme est assumée, mesurée et contrôlée automatiquement.

7. Intégration de l'intelligence artificielle

7.1 Cycle de vie des modèles

Deux modèles d'apprentissage automatique sont intégrés à l'architecture : un modèle de détection de fraude, qui note chaque paiement en temps réel, et un modèle de recommandation de produits, qui personnalise l'offre de chaque client. Ils suivent le même cycle de vie, de la collecte des données au réentraînement.

Cycle de vie des modèles d'apprentissage automatique

Détection de fraude et recommandation de produits : de la collecte des données au réentraînement

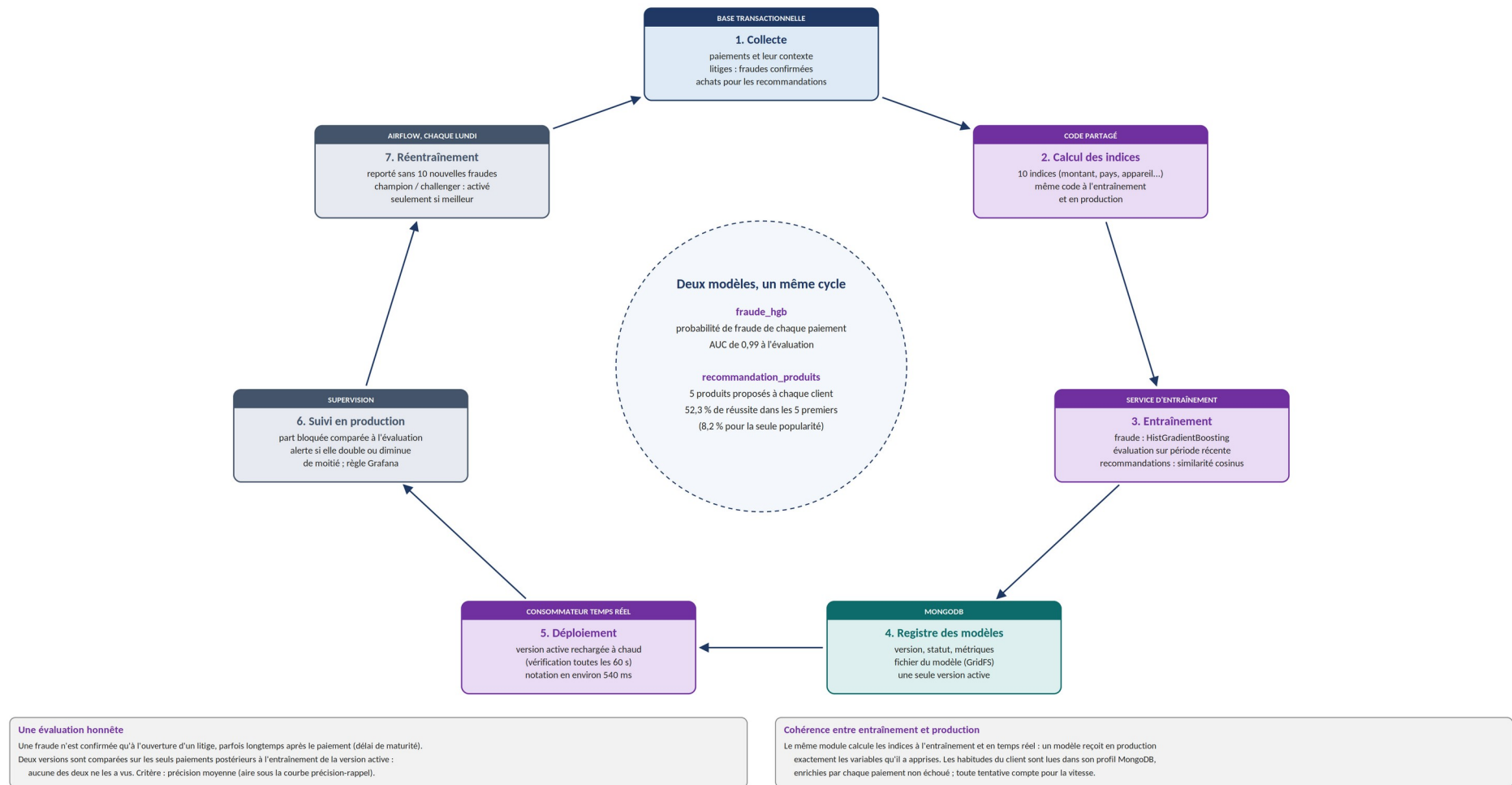


Figure 7 - Cycle de vie des modèles d'apprentissage automatique

Comment lire ce diagramme

Le cycle se lit dans le sens des aiguilles d'une montre, en partant du haut. Les couleurs reprennent celles des couches de l'architecture. Au centre figurent les résultats des deux modèles ; en bas, les deux garanties qui rendent le cycle fiable.

7.2 Intégration dans la base NoSQL

L'énoncé demande d'intégrer les modèles « dans le système NoSQL ». Les modèles ne s'exécutent pas à l'intérieur de MongoDB : ils sont servis par le consommateur temps réel. MongoDB joue trois rôles :

- **magasin de variables** (feature store) : le profil de chaque client conserve ses habitudes (pays, appareils et marchands connus, montants, horodatages récents), lues et mises à jour à chaque paiement ;
- **registre des modèles** : chaque version est décrite dans `modeles_ml` (statut, seuils, métriques, période des données, versions des bibliothèques) et son fichier est stocké dans GridFS ;
- **stockage des prédictions** : le score, la décision et les indices de chaque paiement sont conservés dans sa fiche enrichie, et les recommandations dans le profil du client.

7.3 Extraction des indices

Dix indices sont calculés pour chaque paiement. Le **même module** (`indicateurs.py`) les calcule à l'entraînement, en rejouant l'historique dans l'ordre chronologique, et en production : le modèle reçoit en temps réel exactement les variables qu'il a apprises.

Indice	Signal recherché
montant_eur_log	Montant en euros, sur une échelle logarithmique
ecart_au_montant_moyen	Montant inhabituel pour ce client
pays_ip_inhabituel	Paiement depuis un pays jamais utilisé par le client
appareil_inhabituel	Type d'appareil jamais utilisé par le client
marchand_nouveau	Premier paiement chez ce marchand
nb_paiements_derniere_heure	Rafale de tentatives (échecs compris)
heure_est_nuit	Moment du paiement
anciennete_jours	Ancienneté du compte client
nb_paiements_anterieurs	Historique disponible pour juger des habitudes

7.4 Détection de fraude en temps réel

Élément	Choix
Algorithme	HistGradientBoosting : ensemble d'arbres de décision construits successivement, chacun corrigeant les erreurs des précédents ; pondération des classes pour compenser la rareté des fraudes
Étiquettes	Une fraude est confirmée par un litige pour motif de fraude

Élément	Choix
Évaluation	Première version : découpage temporel, 80 % des paiements les plus anciens pour apprendre, 20 % les plus récents pour évaluer ; AUC de 0,99
Décision	Moins de 0,5 : accepter ; de 0,5 à moins de 0,8 : vérifier (authentification forte ou revue manuelle) ; 0,8 et plus : bloquer
Mise en service	Le consommateur vérifie toutes les 60 secondes la version active et la recharge à chaud, sans interruption du flux

L'efficacité se lit dans l'entrepôt : parmi les paiements bloqués, 25,7 % ont donné lieu à un litige pour fraude, contre 0,04 % parmi les paiements acceptés (chapitre 10, question 7).

7.5 Personnalisation et analyses prédictives

Le modèle de recommandation repose sur un filtrage collaboratif produit à produit : deux produits sont proches s'ils sont souvent achetés par les mêmes clients (similarité cosinus). Pour chaque client, les cinq produits les plus proches de ses achats, et pas encore achetés, sont recommandés et mis à jour à chaque paiement.

Indicateur d'évaluation	Modèle	Référence : popularité seule
Taux de réussite dans les 5 premières recommandations	52,3 %	8,2 %
Couverture du catalogue	100 %	-

Les analyses prédictives s'appuient sur les mêmes données : la probabilité de fraude de chaque paiement, la segmentation des clients par récence, fréquence et montant (qui repère par exemple les clients « À risque » de départ), et les tendances hebdomadaires du chiffre d'affaires calculées dans l'entrepôt.

7.6 Suivi et mise à jour des modèles

Mécanisme	Fonctionnement
Suivi de la dérive	Chaque semaine, la part de paiements bloqués en production est comparée à celle de l'évaluation ; une alerte est consignée si elle double ou diminue de moitié. Grafana surveille aussi cette part en continu
Réentraînement	Chaque lundi, Airflow appelle le service d'entraînement ; le réentraînement de la fraude est reporté tant que 10 nouvelles fraudes confirmées ne sont pas disponibles
Champion / challenger	Une nouvelle version n'est activée que si elle fait mieux que la version en service sur des paiements postérieurs à l'entraînement de celle-ci, jamais vus par aucune des deux ; sinon, elle reste candidate
Traçabilité	Chaque version reste dans le registre avec ses métriques ; une seule version active par modèle, garantie par un index unique partiel

En démonstration, la dérive a été détectée : 20,3 % de paiements bloqués en production contre 1,2 % à l'évaluation. Cet écart est attendu, car le générateur simule une part de fraude plus élevée en temps réel ; il

illustre le fonctionnement de l'alerte. Une version candidate du modèle de recommandation a par ailleurs été entraînée puis conservée comme candidate, faute d'amélioration : la règle champion / challenger a joué son rôle.

En synthèse

Les modèles sont intégrés de bout en bout : variables calculées par un code partagé, registre et prédictions dans MongoDB, notation en temps réel avec rechargement à chaud, suivi de la dérive et réentraînement automatique sous contrôle de la règle champion / challenger.

8. Sécurité et conformité

8.1 Cadre de sécurité

Le cadre de sécurité repose sur la défense en profondeur : plusieurs couches de protection superposées, de sorte que la défaillance de l'une soit compensée par les suivantes. Il couvre les trois exigences de l'énoncé : chiffrement, contrôle d'accès par rôles et journalisation d'audit.

Sécurité et conformité

Qui accède à quoi (moindre privilège), et comment les données sont protégées (défense en profondeur)

Qui accède à quoi : un compte par programme, un rôle par usage

Compte (programme ou personne)	Base transactionnelle (OLTP)	Entrepôt analytique (OLAP)	Base NoSQL (MongoDB)
svc_generateur <i>application de paiement</i>	role_application_paiements crée paiements, clients, remboursements et litiges ; ni modification ni suppression	aucun accès	role_generateur_nosql insère navigation, avis, journaux, documents ; aucun accès aux paiements ni aux profils
svc_flux <i>consommateur, entraînement</i>	role_flux_temps_reel lit les paiements (sans adresse IP) et le pays des clients, jamais l'e-mail ; écrit le seul score	aucun accès	role_flux_nosql lit et écrit fiches, profils et modèles ; aucune suppression
svc_airflow, svc_airflow_dwh <i>orchestrateur</i>	role_orchestrateur lecture seule (e-mail chiffré, sans la clé) et statistiques de réplication	role_chargement écrit faits, dimensions et agrégats ; vues rafraichies par fonction dédiée	role_orchestrateur_nosql lit journaux et index pour les contrôles ; consigne leur résultat
cdc_debezium <i>capture des changements</i>	réplication et lecture de la seule table des paiements ; adresse IP exclue du flux	aucun accès	aucun accès
replicator <i>réplica</i>	réplication physique uniquement, connexion chiffrée	aucun accès	aucun accès
svc_grafana, svc_grafana_dwh <i>supervision</i>	role_supervision statistiques internes ; date, statut et score des paiements seulement	role_supervision suivi des chargements et dates de chargement seulement	aucun accès
analyste_demo <i>analyste</i>	aucun accès	role_analyste faits, dimensions, agrégats et vues ; pas de journal d'audit	aucun accès
auditeur_demo <i>auditeur</i>	aucun accès	role_auditeur journal d'audit et suivi des chargements ; pas de paiements ni de clients	aucun accès

Comptes administrateurs réservés à la création des bases et des rôles, jamais utilisés par les programmes. Comptes svc_ et de supervision : mots de passe aléatoires de 48 caractères.

Références réglementaires couvertes

RGPD : article 5 (minimisation, limitation de la conservation des journaux à 90 jours), article 25 (protection dès la conception : adresse IP exclue du flux), article 32 (sécurité du traitement : chiffrement, contrôle d'accès, traçabilité)

PCI-DSS : exigence 3 (aucun numéro de carte stocké), exigence 4 (chiffrement des transmissions), exigence 7 (restriction des accès), exigence 8 (identifiants protégés), exigence 10 (journalisation et surveillance)

Comment les données sont protégées

1. Réseau et transport

Chiffrement TLS 1.3 obligatoire vers la base transactionnelle, serveur vérifié (verify-full) ; connexions en clair refusées
2. Authentification

Comptes dédiés, mots de passe vérifiés par SCRAM-SHA-256 ; secrets hors du code et du dépôt (fichier .env exclu de Git)
3. Autorisation

Contrôle d'accès par rôles : droits par table, par colonne et par collection ; aucun compte de service ne peut supprimer dans la caisse ni MongoDB
4. Données

E-mails chiffrés (pgcrypto), cartes remplacées par un jeton, minimisation dans l'entrepôt, adresse IP hors du flux
5. Traçabilité

Journal d'audit non modifiable (déclencheurs), copié dans l'entrepôt, lisible par le seul auditeur
6. Contrôle continu

13 contrôles quotidiens (rapport daté), 5 règles d'alerte, 45 tests automatisés des droits et du chiffrement

de la couche la plus extérieure (le réseau) à la plus intérieure (le contrôle continu)

Figure 8 - Sécurité et conformité : matrice des accès et couches de protection

Comment lire ce diagramme

Dans la matrice, une case colorée indique le rôle et ses droits sur la base de la colonne ; une case blanche « aucun accès » signifie que le compte ne peut pas se connecter à cette base. Les nombreuses cases blanches traduisent le moindre privilège. À droite, les six couches se lisent de la plus extérieure (le réseau) à la plus intérieure (le contrôle continu) : si une couche cède, les suivantes protègent encore.

8.2 Chiffrement

Données	Protection	Détail
Échanges avec la base transactionnelle	TLS 1.3 obligatoire	Certificat signé par une autorité propre au projet ; vérification complète par les programmes (verify-full) : chiffrement, autorité et nom du serveur ; connexions en clair refusées par la base
Adresses e-mail des clients	Chiffrement symétrique (pgcrypto)	Stockées chiffrées ; la clé n'est transmise à aucun programme d'analyse
Numéros de carte	Tokenisation	Seuls un jeton, la marque et les quatre derniers chiffres sont conservés ; aucun numéro complet n'est stocké
Mots de passe des comptes	SCRAM-SHA-256	Mots de passe aléatoires de 48 caractères pour les comptes de service et de supervision
Secrets	Variables d'environnement	Fichier <code>.env</code> exclu du dépôt Git ; aucun secret dans le code ni dans la configuration du connecteur

Trois tests automatisés le prouvent : une connexion chiffrée et vérifiée est acceptée en TLS 1.3 ; une connexion sans chiffrement est refusée par la base ; une connexion vers l'adresse IP du serveur, absente du certificat, est refusée par le programme lui-même, ce qui protège contre l'usurpation de serveur.

8.3 Contrôle d'accès par rôles

Chaque programme utilise son propre compte, doté d'un rôle limité au strict nécessaire. Le rôle porte les droits, le compte porte l'identité : un nouveau programme reçoit un nouveau compte et le rôle existant.

Base	Rôle	Droits	Interdits notables
Transactionnelle	role_application_paiements	Créer marchands, clients, paiements, remboursements, litiges	Aucune modification ni suppression
Transactionnelle	role_flux_temps_reel	Lire les paiements sans l'adresse IP, le pays des clients ; écrire le seul score	Lire l'e-mail, modifier un statut, créer un paiement
Transactionnelle	role_orchestrateur	Lecture seule, statistiques de réplication	Toute écriture
Transactionnelle	role_supervision	Statistiques internes ; date, statut et score des paiements	Montants, données des clients
Entrepôt	role_chargement	Écrire faits, dimensions, agrégats ; rafraîchir les vues	Supprimer une table

Base	Rôle	Droits	Interdits notables
Entrepôt	role_analyste	Lire faits, dimensions, agrégats et vues	Journal d'audit, toute écriture
Entrepôt	role_auditeur	Lire le journal d'audit et le suivi des chargements	Paielements, clients
NoSQL	rôles par programme	Opérations définies collection par collection	Aucune suppression pour aucun compte de service

Les droits sont définis jusqu'au niveau de la **colonne** dans PostgreSQL et de la **collection** dans MongoDB. Quarante-deux tests automatisés tentent, sous l'identité de chaque compte, des opérations autorisées et interdites : 18 sur la base transactionnelle, 11 sur l'entrepôt et 13 sur MongoDB, tous conformes.

8.4 Journalisation d'audit

Toute création, modification ou suppression sur les tables sensibles de la base transactionnelle (marchands, clients, moyens de paiement, abonnements, paiements) est enregistrée par déclencheur dans le journal d'audit, avec les valeurs avant et après, le compte et la date. Le journal est non modifiable : toute mise à jour ou suppression est refusée par la base. Il est copié toutes les 15 minutes dans l'entrepôt, sans les valeurs, où seul le rôle d'auditeur peut le consulter.

8.5 Conformité réglementaire

Exigence	Réponse	Contrôle automatique
RGPD, article 5 : minimisation	Entrepôt sans donnée personnelle directe ; adresse IP absente du flux	Oui
RGPD, article 5 : limitation de la conservation	Journaux techniques supprimés après 90 jours (index d'expiration)	Oui
RGPD, article 25 : protection dès la conception	Adresse IP exclue de la capture des changements	Oui
RGPD, article 32 : sécurité du traitement	Chiffrement des e-mails et des échanges, contrôle d'accès, audit	Oui
PCI-DSS, exigence 3	Aucun numéro de carte complet stocké	Oui
PCI-DSS, exigence 4	Transmissions chiffrées en TLS 1.3	Oui
PCI-DSS, exigences 7 et 8	Moindre privilège ; identifiants hors du code	Oui
PCI-DSS, exigence 10	Journal d'audit actif et non modifiable	Oui
CCPA : droits de savoir et de suppression	Identifiant client commun aux trois bases, permettant de retrouver toutes les données d'une personne	Procédure

Les réglementations peuvent entrer en tension : le droit à l'effacement (RGPD, CCPA) s'oppose à l'obligation de conserver les opérations financières pendant plusieurs années (lutte contre le blanchiment). La réponse retenue est la **pseudonymisation** : à la demande d'une personne, ses données d'identification

(adresse e-mail chiffrée, profil et historique de navigation) sont supprimées, tandis que ses paiements sont conservés sous un identifiant technique qui ne permet plus de l'identifier. L'entrepôt, qui ne contient aucune donnée personnelle directe, n'a pas à être modifié.

8.6 Rapport de conformité automatisé

La chaîne Airflow `contrôles_conformite` exécute chaque jour treize contrôles et produit un rapport daté au format Markdown. Elle échoue volontairement si un seul contrôle n'est pas conforme, ce qui rend l'écart immédiatement visible.

Domaine	Contrôle	Référence
Base transactionnelle	Chiffrement des échanges réseau (TLS)	RGPD art. 32 ; PCI-DSS exig. 4
Base transactionnelle	Chiffrement des adresses e-mail des clients	RGPD art. 32
Base transactionnelle	Absence de numéro de carte complet	PCI-DSS exig. 3
Base transactionnelle	Moindre privilège du compte de capture des changements	PCI-DSS exig. 7
Base transactionnelle	Journal d'audit actif et non modifiable	PCI-DSS exig. 10
Entrepôt analytique	Minimisation des données personnelles	RGPD art. 5.1.c
Capture des changements	Exclusion de l'adresse IP du flux temps réel	RGPD art. 25
Capture des changements	Absence de secret en clair dans la configuration	PCI-DSS exig. 8
Capture des changements	Disponibilité du connecteur	Continuité du service
Base NoSQL	Durée de conservation limitée des journaux techniques	RGPD art. 5.1.e
Base NoSQL	Règles de validation des collections	Qualité et intégrité
Disponibilité	Réplication de la base transactionnelle	Reprise après sinistre
Disponibilité	Retard de la capture des changements	Fraîcheur des données

Lors des mesures, les treize contrôles étaient conformes. La surveillance en temps réel des failles de sécurité est assurée par la supervision (chapitre 9) : les connexions non chiffrées et la liste des comptes connectés y sont affichées en continu, et l'interruption de la capture des changements déclenche une alerte. L'extension de ces alertes aux connexions et aux comptes est prévue dans l'architecture cible (chapitre 11).

En synthèse

Chiffrement, contrôle d'accès par rôles et journal d'audit sont mis en œuvre et prouvés par 45 tests automatisés. La conformité au RGPD, à PCI-DSS et à la CCPA est vérifiée chaque jour par 13 contrôles, consignés dans un rapport daté.

9. Supervision, performances et évolutivité

9.1 Tableau de bord de supervision

Un tableau de bord Grafana surveille en continu la santé de l'infrastructure, toutes les 30 secondes. Il est entièrement décrit par des fichiers versionnés (sources de données, tableau de bord, règles d'alerte). Il interroge la base transactionnelle et l'entrepôt avec deux comptes dédiés en lecture seule, sans accès aux montants ni aux données des clients, et par une connexion chiffrée et vérifiée vers la base transactionnelle.

Zone	Indicateurs	Problème détecté
Flux de paiements	Paielements par minute, dont bloqués ; part bloquée sur une heure ; paiements en attente de score	Chute d'activité, vague de fraude, dérive, consommateur arrêté
Réplication et capture	Retard du réplica ; retard de lecture de Debezium ; état de sa connexion ; connexions non chiffrées ; comptes connectés	Risque de perte, journal qui s'accumule, capture interrompue, connexion en clair
Entrepôt	Minutes depuis le dernier passage ; paiements relus ; paiements chargés par heure	Chaîne de chargement arrêtée ou incomplète
Performances	Lectures servies par le cache ; taille des bases	Mémoire insuffisante, besoin de capacité



Figure 9 - Tableau de bord de supervision : flux de paiements et réplication

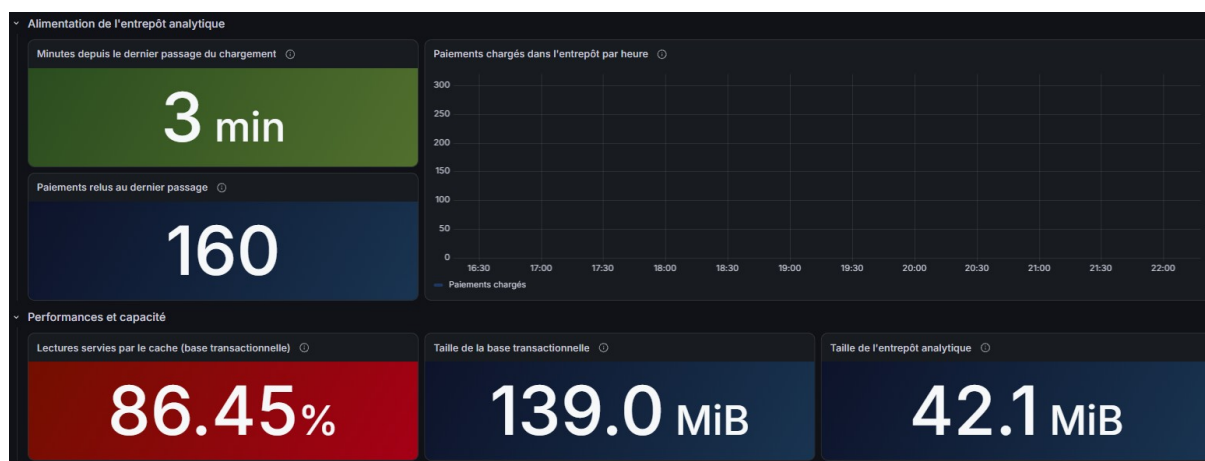


Figure 10 - Tableau de bord de supervision : entrepôt et performances

9.2 Règles d'alerte

Cinq règles d'alerte sont évaluées chaque minute, même lorsque personne ne consulte le tableau de bord. Une durée de persistance évite les alertes passagères ; l'absence de résultat déclenche elle-même l'alerte lorsqu'elle signale une panne (slot ou réplica disparus).

Règle	Condition	Persistance	Gravité
Capture des changements interrompue	Debezium déconnecté de son slot	2 minutes	Critique
Entrepôt analytique non alimenté	Plus de 35 minutes sans passage du chargement	1 minute	Avertissement
Paielements sans score de fraude	Plus de 20 paielements récents en attente	2 minutes	Critique
Dérive du modèle de fraude	Plus de 15 % de paielements bloqués sur une heure (à partir de 50 paielements)	10 minutes	Avertissement
Réplica en retard ou déconnecté	Plus de 100 Mo de retard, ou réplica absent	5 minutes	Critique

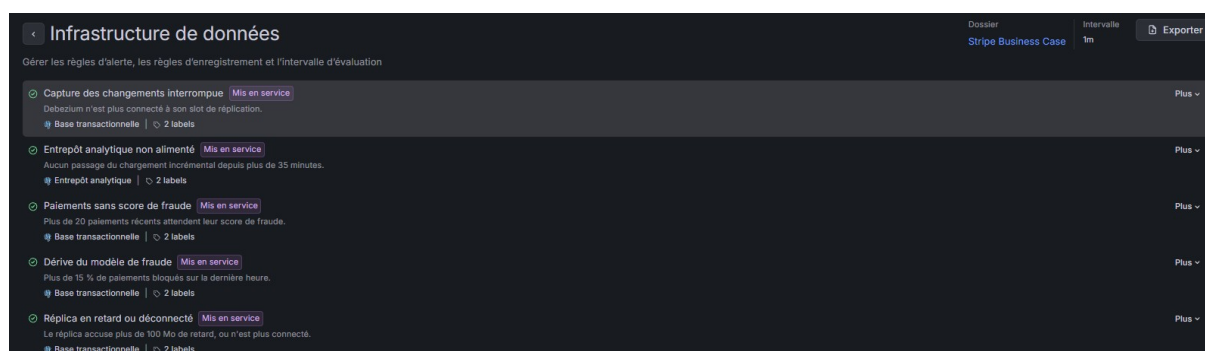


Figure 11 - Règles d'alerte chargées dans Grafana

9.3 Identifier et optimiser : un exemple mesuré

La supervision a mis en évidence un taux de lecture en cache de 85,66 % sur la base transactionnelle, sous le seuil de 90 %. La cause était mesurable : la base occupait 139 Mio, alors que la mémoire de cache de

PostgreSQL (`shared_buffers`) était restée à sa valeur par défaut de 128 Mo. Elle a été portée à 512 Mo et les statistiques ont été remises à zéro pour mesurer l'effet. Juste après le redémarrage, le taux reste bas, la mémoire de cache étant vide : il se mesure après une période d'activité. Cette démarche illustre la gestion proactive attendue : surveiller, détecter, optimiser, puis vérifier.

9.4 Stratégies de performance

Stratégie	Base transactionnelle	Entrepôt	Base NoSQL
Index	13 index, dont 3 partiels	Index sur chaque clé étrangère, index partiel sur les suspects	Index composés, partiels, plein texte, unique partiel, expiration
Partitionnement	Table des paiements partitionnable par mois (cible) ; flux Kafka réparti par client	Partitions mensuelles de la table de faits	Clé de répartition par client (cible)
Mise en cache	Mémoire de cache portée à 512 Mo	Agrégats et vues matérialisées : résultats pré-calculés	Profil client : habitudes pré-calculées, lues en une seule lecture

9.5 Montée en charge horizontale

L'énoncé demande une architecture capable de « monter en charge horizontalement », c'est-à-dire en ajoutant des machines plutôt qu'en agrandissant une seule machine. Chaque composant le permet sans changement de conception :

Composant	Mécanisme de montée en charge	Préparation dans le projet
Kafka et consommateur	Ajout de partitions et d'instances du consommateur dans le même groupe ; chaque instance traite une part des partitions	Sujet à 3 partitions ; clé client garantissant l'ordre quelle que soit l'instance
Base NoSQL	Répartition des données (sharding) sur plusieurs serveurs, avec réplication de chaque partie	Clé naturelle de répartition : identifiant du client, présent dans les documents principaux
Base transactionnelle	Réplicas en lecture pour les analyses opérationnelles ; base SQL distribuée à très grande échelle	Réplica en place ; requêtes d'analyse déjà déportées vers l'entrepôt
Entrepôt	Entrepôt en colonnes et calcul distribué	Modèle en étoile et partitionnement transposables tels quels
Airflow	Exécuteur distribué (plusieurs travailleurs)	Tâches indépendantes et idempotentes

10. Requêtes métier

10.1 Questions et bases interrogées

Dix-sept requêtes montrent comment les modèles de données répondent aux questions métier : chiffre d'affaires, fraude, segmentation, personnalisation et suivi. Chacune est posée à la base la mieux adaptée. Elles sont rangées dans le dossier `09_requetes_metier` du dépôt, commentées, et ont été exécutées sur les données de démonstration.

N°	Question	Base	Technique démontrée
1	Chiffre d'affaires net par mois et par région	Entrepôt	Jointures en étoile
2	Évolution hebdomadaire du chiffre d'affaires	Entrepôt	Série temporelle, fonctions de fenêtre
3	Dix marchands les plus rentables et leur taux de remboursement	Entrepôt	Table pré-agrégée
4	Pays de l'adresse IP les plus exposés à la fraude	Entrepôt	Vue matérialisée
5	Produits les plus vendus et les plus remboursés	Entrepôt	Vue matérialisée
6	Segmentation des clients et poids de chaque segment	Entrepôt	Segmentation RFM, fenêtre sur l'ensemble
7	Efficacité du modèle de fraude	Entrepôt	Croisement des décisions et des litiges
8	Derniers paiements d'un client, avec remboursements et litige	Transactionnelle	Jointures normalisées, index
9	Marchands au taux de litige anormal	Transactionnelle	Surveillance opérationnelle
10	Historique des actions sur un paiement	Transactionnelle	Journal d'audit
11	Paiements bloqués dans l'heure et indices déclenchés	NoSQL	Pipeline d'agrégation, sous-documents
12	Décisions et latence du flux sur 24 heures	NoSQL	Médiane, regroupement
13	Recommandations du client le plus actif	NoSQL	Personnalisation
14	Entonnoir de conversion des visites	NoSQL	Listes imbriquées
15	Note moyenne par marchand et avis sur la livraison	NoSQL	Agrégation, recherche plein texte
16	Composition des dossiers de litige	NoSQL	XML, fichiers GridFS, références
17	Un même paiement dans les trois bases	Les trois	Intégration par identifiants communs

10.2 Analyse du chiffre d'affaires

La question 2 illustre l'analyse de séries temporelles : le chiffre d'affaires net de chaque semaine, sa moyenne glissante sur quatre semaines et sa variation d'une semaine à l'autre.

```
WITH semaines AS (  
    SELECT date_trunc('week', d.date_complete)::date AS semaine,  
           sum(f.montant_eur) FILTER (WHERE f.statut <> 'echouee')  
             - sum(f.montant_rembourse_eur) AS ca_net_eur  
    FROM fait_transactions f  
    JOIN dim_date d ON d.date_cle = f.date_cle  
    GROUP BY 1  
)  
SELECT semaine,  
       round(ca_net_eur, 2) AS ca_net_eur,  
       round(avg(ca_net_eur) OVER (ORDER BY semaine  
                                   ROWS BETWEEN 3 PRECEDING AND CURRENT ROW), 2) AS moyenne_glissante_4_semaines,  
       round(100.0 * (ca_net_eur - lag(ca_net_eur) OVER w)  
             / NULLIF(lag(ca_net_eur) OVER w, 0), 1) AS variation_pct  
FROM semaines  
WINDOW w AS (ORDER BY semaine)  
ORDER BY semaine DESC  
LIMIT 12;
```

Sur les données de démonstration, le chiffre d'affaires net hebdomadaire passe d'environ 150 000 euros en juillet à des semaines comprises entre 158 000 et 212 000 euros en septembre. La dernière semaine affiche une hausse anormale, due au rythme accéléré du générateur en mode continu (chapitre 11).

10.3 Détection de fraude

La question 7 croise la décision du modèle et les litiges pour fraude enregistrés ensuite :

Décision du modèle	Paielements	Litiges pour fraude	Part
Bloquer (score de 0,8 et plus)	1 618	416	25,71 %
Vérifier (0,5 à moins de 0,8)	372	12	3,23 %
Accepter (moins de 0,5)	45 057	16	0,04 %

Les paiements bloqués concentrent les fraudes confirmées. La part est plus faible pour les paiements récents, dont les litiges ne sont pas encore arrivés (délai de maturité des étiquettes). La question 4 montre de son côté que le Japon, Singapour et l'Inde présentent la plus forte part de paiements suspects parmi les pays d'adresse IP.

En temps réel, la question 11 explique chaque blocage à partir des indices imbriqués dans la fiche du paiement :

```

db.transactions_enrichies.aggregate([
  { $match: { "score.decision": "bloquer", date_heure: { $gte: depuisUneHeure } } },
  { $sort: { date_heure: -1 } },
  { $limit: 10 },
  { $project: {
    montant: 1, devise: 1, score: "$score.valeur", pays_ip: "$contexte.pays_ip",
    indices: { $concatArrays: [
      { $cond: [{ $eq: ["$indicateurs.pays_ip_inhabituel", 1] },
        ["pays IP inhabituel"], [] ] },
      { $cond: [{ $eq: ["$indicateurs.marchand_nouveau", 1] },
        ["marchand nouveau"], [] ] },
      { $cond: [{ $gte: ["$indicateurs.nb_paiements_derniere_heure", 2] },
        ["rafale"], [] ] },
      { $cond: [{ $gte: ["$indicateurs.ecart_au_montant_moyen", 2] },
        ["montant inhabituel"], [] ] }
    ] }
  } }
])

```

Exemple de résultat : trois paiements depuis l'Inde, bloqués avec un score de 0,98, cumulaient un pays d'adresse IP jamais utilisé, un marchand nouveau et un montant dix à onze fois supérieur aux habitudes du client, profil typique d'une carte volée.

10.4 Segmentation et personnalisation

Segment (question 6)	Clients	Part du chiffre d'affaires
Champions	489	55,5 %
Fidèles	450	18,8 %
À risque	259	11,7 %
Inactifs	541	11,5 %
Nouveaux	259	2,6 %

Les Champions, soit un quart des clients, réalisent plus de la moitié du chiffre d'affaires ; les clients « À risque » sont la cible prioritaire des actions de fidélisation. La question 13 restitue les cinq recommandations calculées pour un client, et la question 14 mesure l'entonnoir de conversion des visites : 94,9 % consultent une page, 50,0 % ajoutent un article au panier et 45,2 % paient.

10.5 Requête transverse : un même paiement dans les trois bases

La question 17 suit un paiement bloqué à travers les trois bases, grâce à son identifiant commun. Chaque base apporte ce qu'elle seule sait :

Base	Informations apportées sur le paiement 613c0ea8
Transactionnelle	Marchand (jeux vidéo), pays du client (Brésil), moyen de paiement (portefeuille Google Pay), statut, montant de 131,77 euros, score de 0,8298
NoSQL	Décision « bloquer » prise en 711 millisecondes, dix indices calculés à l'instant du paiement, historique des statuts

Base	Informations apportées sur le paiement 613c0ea8
Entrepôt	Jour de la semaine (dimanche), région du marchand (Europe), pays de l'adresse IP (Belgique), montant en euros, chargement 13 minutes après le paiement

En synthèse

Les dix-sept requêtes démontrent que chaque modèle répond aux questions pour lesquelles il a été conçu, et que l'intégration par identifiants communs permet de suivre une même donnée d'une base à l'autre.

11. Limites et architecture cible

11.1 Limites de la démonstration

Limite	Effet	Réponse ou évolution
Rythme accéléré du générateur	Un paiement toutes les deux secondes, contre une centaine par jour dans l'historique : la semaine en cours paraît anormalement forte et presque tous les clients semblent récents	Lecture des analyses sur les mois complets ; générateur arrêté hors démonstration ; rythme réaliste en production
Empoisonnement du profil	Un paiement frauduleux non échoué enrichit les habitudes du client : son pays ou son appareil devient « connu »	N'enrichir le profil qu'avec les paiements acceptés et non contestés
Poste unique	Kafka, les bases et Airflow tournent sur une seule machine, sans redondance	Grappes de serveurs dans l'architecture cible (section 11.2)
Bascule manuelle	La promotion du réplica est une opération manuelle	Gestionnaire de haute disponibilité (Patroni) et slot de capture synchronisé sur le réplica
Chiffrement partiel des échanges	Seule la base transactionnelle impose le TLS	TLS pour Kafka, MongoDB et l'entrepôt ; chiffrement des disques
Alertes sans notification	Les alertes sont visibles dans Grafana mais n'envoient pas de message	Courriel ou messagerie d'équipe ; alertes sur les connexions et comptes inattendus
Score après enregistrement	Le score arrive environ 0,5 seconde après l'enregistrement du paiement : un paiement « bloqué » est déjà enregistré	Appel synchrone du modèle par l'application avant validation, la capture servant à l'enrichissement

11.2 Architecture cible dans le cloud

L'architecture se transpose dans le cloud sans changement de conception : chaque composant a un équivalent en service géré, qui apporte la haute disponibilité, la sauvegarde et la montée en charge.

Composant	Démonstration	Cible (exemple AWS)	Cible (exemple Azure)
Base transactionnelle	PostgreSQL et un réplica	Aurora PostgreSQL multi-zones	Azure Database for PostgreSQL, haute disponibilité
Diffusion	Kafka, un nœud	Amazon MSK, trois courtiers	Event Hubs (protocole Kafka)
Capture des changements	Debezium	Debezium sur MSK Connect	Debezium sur conteneurs
Base NoSQL	MongoDB, un nœud	MongoDB Atlas, grappe répartie	MongoDB Atlas ou Cosmos DB (API MongoDB)
Entrepôt	PostgreSQL en étoile	Amazon Redshift	Azure Synapse Analytics

Composant	Démonstration	Cible (exemple AWS)	Cible (exemple Azure)
Orchestration	Airflow	Amazon MWAA	Airflow sur Kubernetes (AKS)
Modèles	Service FastAPI et registre MongoDB	SageMaker (entraînement et registre)	Azure Machine Learning
Supervision	Grafana	Amazon Managed Grafana et CloudWatch	Azure Managed Grafana et Azure Monitor
Secrets	Fichier .env	AWS Secrets Manager	Azure Key Vault

11.3 Serveurs virtuels et grappes de serveurs

Dans l'architecture cible, les traitements de calcul sont exécutés sur des grappes de serveurs (clusters) plutôt que sur une machine unique :

- **conteneurs orchestrés par Kubernetes** (EKS ou AKS) : le consommateur temps réel et le service d'entraînement sont déployés en plusieurs instances, dont le nombre s'ajuste automatiquement à la charge ;
- **serveurs virtuels dédiés à l'entraînement** : des machines plus puissantes sont démarrées le temps d'un réentraînement, puis arrêtées, ce qui limite le coût ;
- **infrastructure décrite en code** (Terraform) : les ressources cloud sont créées et modifiées de façon reproductible, comme les seize services de la démonstration le sont par Docker Compose ;
- **trois zones de disponibilité** : chaque base et chaque courtier Kafka est répliqué sur trois centres de données, pour résister à la perte de l'un d'eux.

En synthèse

La démonstration met en œuvre l'ensemble de l'architecture sur un poste unique, avec des limites identifiées et assumées. L'architecture cible conserve la même conception et la transpose sur des services gérés et des grappes de serveurs, pour la haute disponibilité et la montée en charge.

Annexes

A. Organisation du dépôt

2_Stripe_Business_Case/	
├─ 03_diagrammes/	sept diagrammes et leurs programmes sources
├─ 04_oltp_postgresql/	schéma, règles métier, index, rôles, réplication
├─ 05_olap_entrepot/	schéma en étoile, partitions, vues, rôles
├─ 06_nosql_mongodb/	collections, index, comptes, tests de validation
├─ 07_pipeline/	
│ └─ airflow/	image Airflow du projet
│ └─ dags/	trois chaînes de traitement Airflow
│ └─ debezium/	connecteur de capture des changements
│ └─ generateur/	générateur de données
│ └─ ml/	indices, entraînement, consommateur, suivi
├─ 08_securite_conformite/	certificats, rapports, tests d'accès
├─ 09_requetes_metier/	requêtes des dix-sept questions
├─ 10_captures/	captures d'écran et vidéo
├─ 11_supervision/	tableau de bord et alertes Grafana
├─ docker-compose.yml	description des seize services
└─ .env.example	modèle des variables, sans secret

B. Séquence de déploiement

Étape	Opération	Résultat
1	Génération des secrets (.env) et des certificats TLS	Mots de passe aléatoires, autorité de certification et certificat du serveur
2	Démarrage des bases, de Kafka et de Kafka Connect	Schémas, rôles et collections créés par les scripts d'initialisation
3	Génération de l'historique	Référentiels, marchands, clients, paiements, documents
4	Entraînement initial des deux modèles	Versions 1 actives dans le registre
5	Enregistrement du connecteur Debezium	Capture des changements active
6	Démarrage d'Airflow, du service d'entraînement et de Grafana	Chaînes de traitement, supervision et alertes actives
7	Démarrage du consommateur et du générateur en mode continu	Flux temps réel de bout en bout

C. Glossaire

Terme	Définition
ACID	Atomicité, cohérence, isolation, durabilité : garanties d'une transaction fiable
AUC	Aire sous la courbe ROC : capacité d'un modèle à classer un cas positif au-dessus d'un cas négatif (1 = parfait)

Terme	Définition
Capture des changements (CDC)	Lecture du journal des transactions pour diffuser chaque modification en temps réel
Champion / challenger	Règle selon laquelle une nouvelle version de modèle ne remplace l'ancienne que si elle fait mieux
Cohérence à terme	Les copies d'une donnée rejoignent l'état de la source dans un délai donné
Dérive	Écart entre le comportement d'un modèle en production et celui mesuré à l'évaluation
Dimension	Table qui décrit le contexte d'un fait : date, client, marchand, produit...
Filigrane	Dernière valeur traitée par un chargement incrémental, point de départ du suivant
GridFS	Mécanisme de MongoDB pour stocker des fichiers en blocs, au-delà de 16 Mo
Idempotence	Propriété d'une opération qui produit le même résultat si elle est répétée
OLAP	Traitement analytique : requêtes d'agrégation sur de grands volumes historiques
OLTP	Traitement transactionnel : nombreuses écritures courtes et cohérentes
RFM	Récence, fréquence, montant : méthode de segmentation des clients
Slot de réplication	Marqueur qui oblige PostgreSQL à conserver le journal jusqu'à sa lecture
TLS	Protocole de chiffrement des échanges réseau
Troisième forme normale	Organisation des tables où chaque information n'est stockée qu'une fois